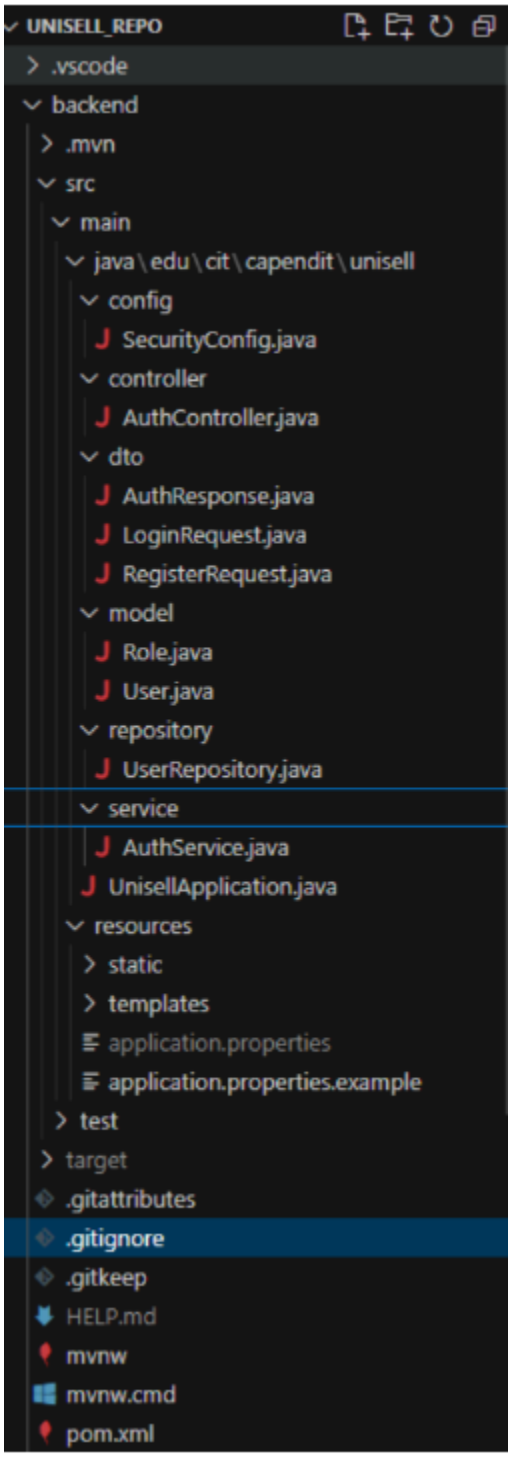
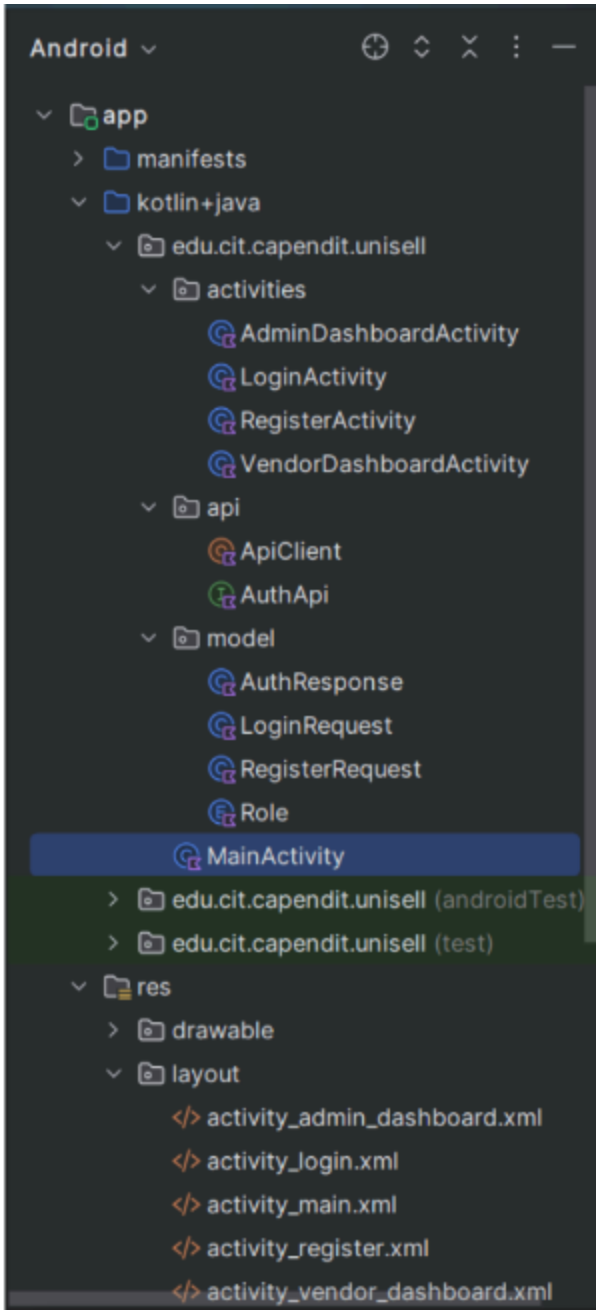
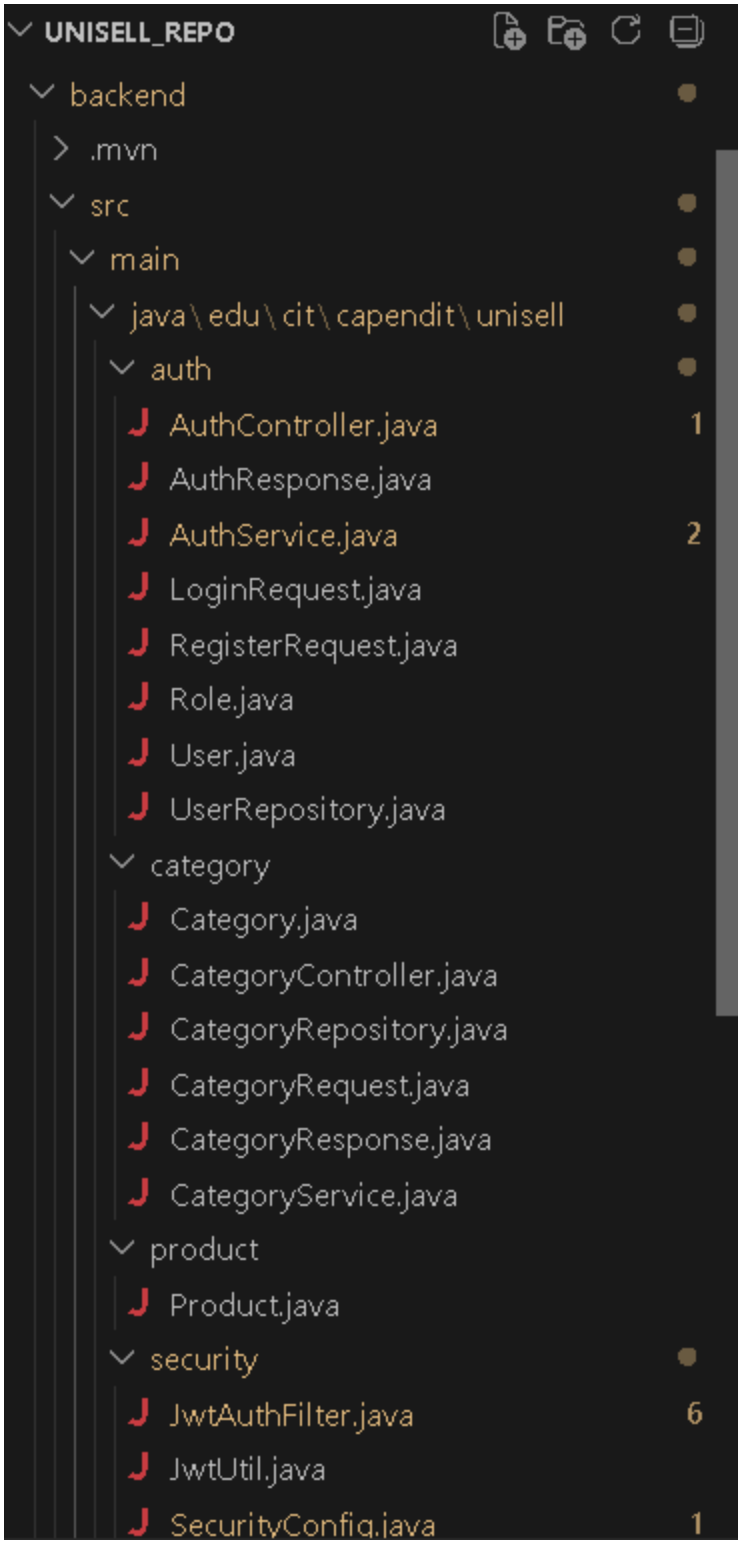
1. **Github Link: https://github.com/suprice4/UniSell_repo**
2. **Refactoring Report**

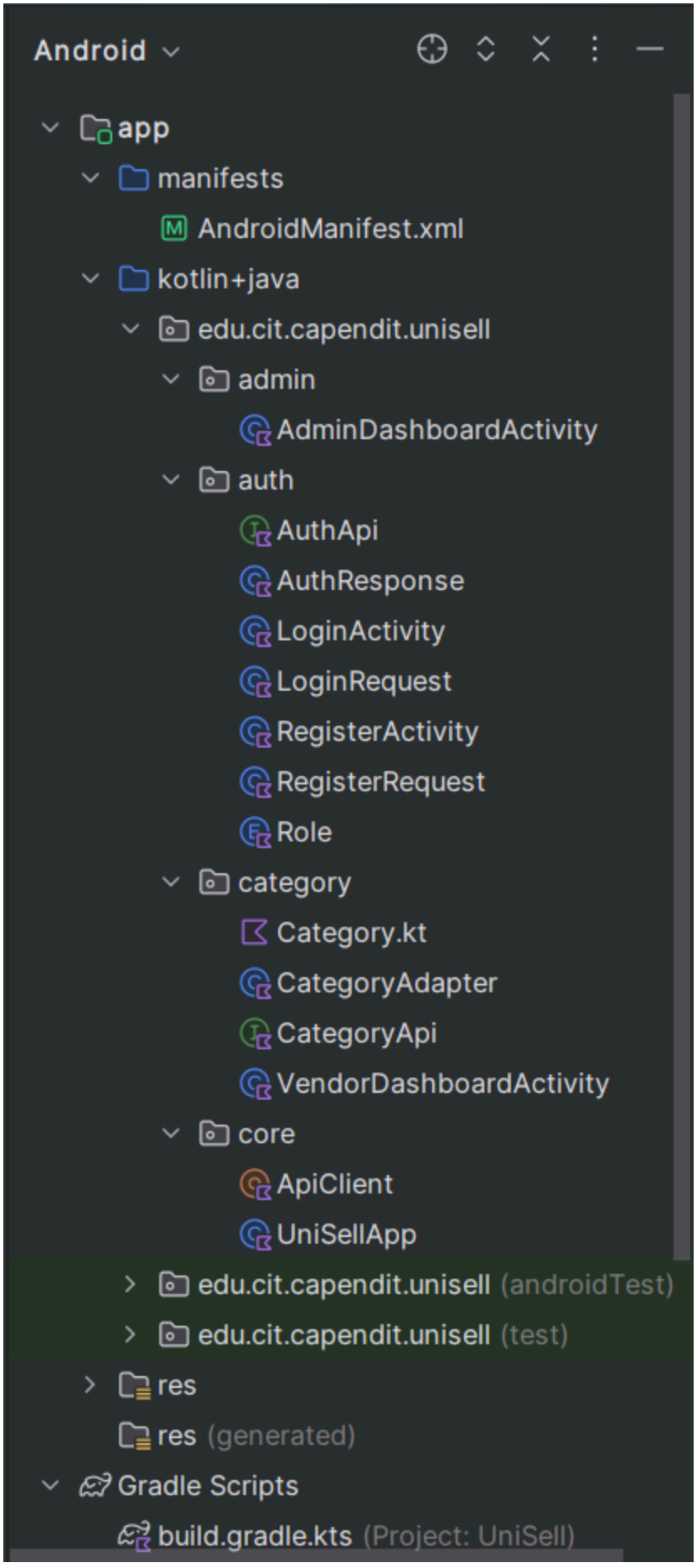
Previous Project Structure





Refactored Project Structure:


UNISELL_REPO
backend
.mvn
src
main
java\edu\cit\capendit\unisell
auth
AuthController.java 1
AuthResponse.java
AuthService.java 2
LoginRequest.java
RegisterRequest.java
Role.java
User.java
UserRepository.java
category
Category.java
CategoryController.java
CategoryRepository.java
CategoryRequest.java
CategoryResponse.java
CategoryService.java
product
Product.java
security
JwtAuthFilter.java 6
JwtUtil.java
SecurityConfig.java 1


Android
app
manifests
AndroidManifest.xml
kotlin+java
edu.cit.capendit.unisell
admin
AdminDashboardActivity
auth
AuthApi
AuthResponse
LoginActivity
LoginRequest
RegisterActivity
RegisterRequest
Role
category
Category.kt
CategoryAdapter
CategoryApi
VendorDashboardActivity
core
ApiClient
UniSellApp
edu.cit.capendit.unisell (androidTest)
edu.cit.capendit.unisell (test)
res
res (generated)
Gradle Scripts
build.gradle.kts (Project: UniSell)

Features converted into Vertical Slices:

Feature	Slice	Components Included
User Registration & Login (Web)	auth/	Entity (User, Role), Repository (UserRepository), DTOs (RegisterRequest, LoginRequest, AuthResponse), Service (AuthService), Controller (AuthController)
Category Management (Web)	category/	Entity (Category), Repository (CategoryRepository), DTOs (CategoryRequest, CategoryResponse), Service (CategoryService), Controller (CategoryController)
Product (in progress) (Web)	product/	Entity (Product) only, repository/service/controller not yet built, intentionally deferred
Security Infrastructure (Web)	security/	JWT utility (JwtUtil), auth filter (JwtAuthFilter), Spring Security config (SecurityConfig) — cross-cutting infrastructure, not a user-facing feature, kept as its own slice since every feature depends on it
User Registration & Login (Android)	auth/	DTOs (RegisterRequest, LoginRequest, AuthResponse), Role enum, AuthApi (Retrofit interface), LoginActivity, RegisterActivity
Category Management (Android)	category/	Category (request/response models), CategoryApi (Retrofit interface), VendorDashboardActivity, CategoryAdapter
Admin Dashboard (Android, placeholder)	admin/	AdminDashboardActivity, placeholder only, no functionality yet, given its own slice ahead of implementation (mirrors backend's product/ precedent)

Shared App/API Infrastructure (Android)	core/	ApiClient (Retrofit/OkHttp setup, JWT interceptors, token storage), UniSellApp (Application class)

Explanation of how each slice works:

- **auth/ (Web)** - handles end-to-end Registration and Login. HTTP requests to AuthController are sent to AuthService, which checks credentials, hashes passwords with BCrypt, and generates JWTs through JwtUtil (imported from security slice). All users are created as VENDOR. The UserRepository and the User/Role entities are in the same package, so there is no need to import between these entities; only the cross-slice dependency on JwtUtil crosses a package boundary.
- **category/ (Web)** - manages CRUD of vendor-owned product categories, fully scaled to the authenticated vendor. CategoryController takes the identity of the vendor from the Spring Security Authentication object (JWT subject) and passes it on to the CategoryService, where it will be used to enforce two invariants: Category names must not be empty or longer than 100 characters, and they must be unique for each vendor, case-insensitive. For each read/update/delete operation, it is scoped by vendorEmail, and if another vendor's categoryID is requested, a 404 will be returned instead of a 403, avoiding the disclosure that the category exists at all. There is a cross-slice dependency between Category.java and User from the auth slice, and this is a perfectly valid dependency with no issues with the pattern.
- **products/ (Web)** - adds a single WIP entity to the product/ folder. It already imports User (auth/) and Category (category/), because a product is owned by a vendor and is categorized under a category. It is now in its own package, ahead of the addition of its repository/service/controller, such that when those layers are added, they are dropped right into the correct slice, rather than having to be reorganized again.
- **security/ (Web)** - infrastructure that is common across all other slices. JwtAuthFilter will run for each request and validate the JWT and fill out the Spring Security context; it will also implement a sliding 30-minute inactivity session by re-issuing a new JWT on each request, where it can see that a token is valid for a user's identity by sending it back with the X-New-Token header. This filter is plugged into the security chain by SecurityConfig, and it specifies the paths that need to be authenticated. This was not included in auth/ because it is not something that a user would interact with directly, but is infrastructure that all features rely on.
- **auth/ (Android)** - LoginActivity/RegisterActivity talking to AuthApi and saving JWT in ApiClient, Redirect to VENDOR DashboardActivity/ADMIN DashboardActivity driven by Role enum
- **category/ (Android)** - renders the category/ UI, with the CategoryAdapter as a driver of the CRUD UI and CategoryApi for the API calls. Renders the category/ UI, with CategoryAdapter acting as a driver of the CRUD UI and CategoryApi as the API calls.
- **admin/ (Android)** — placeholder; own slice created in advance to mirror the changing of the backend product/ precedent.

- core/ (Android) -explicitly parallel to how security/ was justified as a cross-cutting infra on the backend, core/ (Android) — ApiClient as shared cross-cutting infra (JWT interceptor, token refresh via X-New-Token, Retrofit setup), and UniSellApp as the Application class

  Challenges encountered and solutions applied:
  Challenge 1 - Naive package boundaries are violated by cross-slice dependencies.
  While vertical slicing assumes that each feature is self-contained, the features of Category actually depend on User (ownership), and the features of Product depend on both User and Category. If the slices were considered completely separate, then the entities would need to be duplicated, or odd indirection would need to be added.
  Solution: Cross-slice imports were kept explicit and limited – category/ imports User from auth/; product/ imports User and Category from their separate slices. This maintains the direction of the dependency visually in the code and is more honest than the old code's implicit coupling via a shared model/ folder.

  Challenge 2 - Where does cross-cutting infrastructure belong?
  SecurityConfig.java did not behave as a "feature" because it set up security for the whole application and not just one slice.
  Solution: This is embedded in the current security/package as all three are closely related to JWT/authentication infrastructure, without having to create a separate config/package, which would simply distribute related security code to two places.

  Challenge 3 - The remains of a feature that has been deleted.
  SecurityConfig.java had authorization rules for TestController (/*/api/test/admin-only, /api/test/**), that were being removed as part of the clean-up. If these rules were kept after the controller is removed, then a path would be referred to without any controller mapped to it.
  Solution: The two dead rules were eliminated after testing and verifying that this has no effect on behaviour - an unmapped path returns 404, the same regardless of whether or not a security rule mentions it.

  Challenge 4 - Tooling errors during manual refactor when moving files.
  Some move commands were initially unsuccessful, or failures resulted in the files being moved to the wrong place, such as a move command being split over two lines with no destination, resulting in a file being dropped in the repository root instead of the desired package.
  Solution: I used dir /s /b after each move batch to confirm the actual file locations after each move and found the file that had been moved, made the missing destination file folder, and then re-run a single-line move with the corrected file location.

  Challenge 5 - If there's no behavior changed after a “pure” structural refactor, is it a refactor, or is there a real problem?
  The aim was to ensure that the API behaved the same way, which was not enough to be sure by just looking at the code.

Solution: The application compiled successfully after the refactor, and it was started, and the full pre-existing Category verification suite was re-run, and it produced the same status codes and messages as it did before the refactor at all of the endpoints.

Challenge 6 - Stale manifest entry: .item_category was declared as an <activity> in AndroidManifest.xml, but was actually a layout XML resource, not a Kotlin class, which caused a build failure.
Solution: The problem is solved by removing the invalid manifest entry.

Challenge 7 - Environment and tooling incompatibility not related to the refactor itself: system JAVA_HOME: JDK 26, incompatible with the Gradle 8.4 bundled Kotlin DSL compiler: Failed to install a build with Kotlin DSL version 1.6: 26 build failure (opaque java.lang.IllegalArgumentException: 26).
Temporary fix: Set the following property in gradle.properties to point to the bundled JDK in Android Studio.

Challenge 8 - Git/IDE staging interaction: Android Studio's Refactor→Move staged the renames of a file in git, but a later unrelated git add to a single file staged the same rename as well, which wound up with the manifest bugfix commit being bundled with the package move, instead of separate as intended.
Solution: documented, not rewritten through history-editing, so there was little risk of affecting correctness, and rewriting history in a shared student repo seemed to lack value.

Challenge 9 - Failed to get OKHttp's HTTP logging output in Logcat through HttpLoggingInterceptor: Indirect verification of JWT behavior: Logcat did not surface OKHttp's HTTP logging output through HttpLoggingInterceptor.Level.BODY is properly set up and not altered due to the refactor.
Solution: verified JWT attach/refresh indirectly, all authenticated CRUD operations (register, login, category add/edit/delete) succeed and are correctly reflected in the database, which would not be possible without a valid, correctly-attached token.

Screenshots or code samples showing the changes:

Before - model/Category.java:

```
package edu.cit.capendit.unisell.model;

import jakarta.persistence.*;

@Entity
@Table(name = "categories", ...)
public class Category {
    ...
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "vendor_id", nullable = false)
    private User vendor;
    ...
}
```

After - category/Category.java:

```
package edu.cit.capendit.unisell.category;

import edu.cit.capendit.unisell.auth.User;
import jakarta.persistence.*;

@Entity
@Table(name = "categories", ...)
public class Category {
    ...
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "vendor_id", nullable = false)
    private User vendor;
    ...
}
```

Before - service/CategoryService.java (imports section):

```
package edu.cit.capendit.unisell.service;

import edu.cit.capendit.unisell.dto.CategoryRequest;
import edu.cit.capendit.unisell.dto.CategoryResponse;
import edu.cit.capendit.unisell.model.Category;
import edu.cit.capendit.unisell.model.User;
```

```
import edu.cit.capendit.unisell.repository.CategoryRepository;
import edu.cit.capendit.unisell.repository.UserRepository;
```

After - category/CategoryService.java (imports section):

```
package edu.cit.capendit.unisell.category;

import edu.cit.capendit.unisell.auth.User;
import edu.cit.capendit.unisell.auth.UserRepository;
```

Before - activities/LoginActivity.kt (imports section):

```
package edu.cit.capendit.unisell.activities

import edu.cit.capendit.unisell.api.ApiClient
import edu.cit.capendit.unisell.activities.AdminDashboardActivity
import edu.cit.capendit.unisell.model.LoginRequest
```

After - auth/LoginActivity.kt (imports section):

```
package edu.cit.capendit.unisell.auth

import edu.cit.capendit.unisell.core.ApiClient
import edu.cit.capendit.unisell.admin.AdminDashboardActivity
```

Before - api/ApiClient.kt (top of file):

```
package edu.cit.capendit.unisell.api

import android.content.Context
import android.content.SharedPreferences
import okhttp3.OkHttpClient
```

After - core/ApiClient.kt (top of file):

```
package edu.cit.capendit.unisell.core

import android.content.Context
import android.content.SharedPreferences
import edu.cit.capendit.unisell.auth.AuthApi
import edu.cit.capendit.unisell.category.CategoryApi
import okhttp3.OkHttpClient
```

3. **Commit History Table:**

Feature Refactored	**Commit Hash Link**
User Registration & Login (auth/)	https://github.com/suprice4/UniSell_repo/commit/83504f5
Category Management (category/)	https://github.com/suprice4/UniSell_repo/commit/a383fd3
Product entity (product/)	https://github.com/suprice4/UniSell_repo/commit/b45ed00
Security config (security/)	https://github.com/suprice4/UniSell_repo/commit/11f6260
Cleanup - delete TestController & legacy folders	https://github.com/suprice4/UniSell_repo/commit/fe14753
Fix: stale manifest entry (item_category)	https://github.com/suprice4/UniSell_repo/commit/c9c09f4
Fix: JDK 26 / Gradle incompatibility	https://github.com/suprice4/UniSell_repo/commit/df0c7ea
Auth-related classes (Android auth/)	https://github.com/suprice4/UniSell_repo/commit/6064983
Category-related classes (Android category/)	https://github.com/suprice4/UniSell_repo/commit/cd6d2b6
Admin-related classes (Android admin/)	https://github.com/suprice4/UniSell_repo/commit/1f77bb2
Shared API/app infra + cleanup (Android core/)	https://github.com/suprice4/UniSell_repo/commit/847f4f4